

CPEN 416

Lecture 05: Multi-qubit states and gates

Thursday 24 September 2026

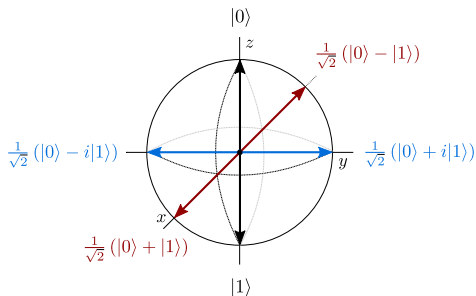
Announcements

- Monday: tutorial assignment 1 (quantum key distribution)
- Quiz 3 on Tuesday
- Midterm in class on Tuesday 6 Oct - details and past midterms available on the weekend in PrairieLearn

Last time

We formalized the idea of *projective measurements*: if we measure a qubit in state $|\varphi\rangle$ with respect to basis $\{|\psi_i\rangle\}$, the probability of observing it in state $|\psi_i\rangle$ (outcome i) is

Visually: geometric projection on the axis represented by a basis.



Last time

To measure in alternative bases, must apply a basis rotation.

Example: measure in Y basis. First, determine mapping between computational basis and other basis:

$$|0\rangle \rightarrow |p\rangle = \frac{1}{\sqrt{2}} (|0\rangle + i|1\rangle)$$

$$|1\rangle \rightarrow |m\rangle = \frac{1}{\sqrt{2}} (|0\rangle - i|1\rangle)$$

At the end of our circuit, apply the reverse (adjoint) to rotate *back* to the computational basis.

Last time

We ran the following circuit and measured in the Y basis.



Learning outcomes

- Mathematically describe a system of multiple qubits
- Describe the action of common multi-qubit gates
- Apply gates controlled on the state of multiple qubits
- Define and give examples of entangled states

Tensor products

Hilbert spaces compose under the *tensor product*.

Let

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad B = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

The tensor product of A and B , $A \otimes B$ is

$$A \otimes B = \begin{pmatrix} a \begin{pmatrix} e & f \\ g & h \end{pmatrix} & b \begin{pmatrix} e & f \\ g & h \end{pmatrix} \\ c \begin{pmatrix} e & f \\ g & h \end{pmatrix} & d \begin{pmatrix} e & f \\ g & h \end{pmatrix} \end{pmatrix} = \begin{pmatrix} ae & af & be & bf \\ ag & ah & bg & bh \\ ce & cf & de & df \\ cg & ch & dg & dh \end{pmatrix}$$

Multi-qubit states

Qubit state vectors also combine under the *tensor product*:

Exercise: Determine the 2-qubit computational basis.

Question: How many n -qubit basis states? (dimension of space?)

Multi-qubit states

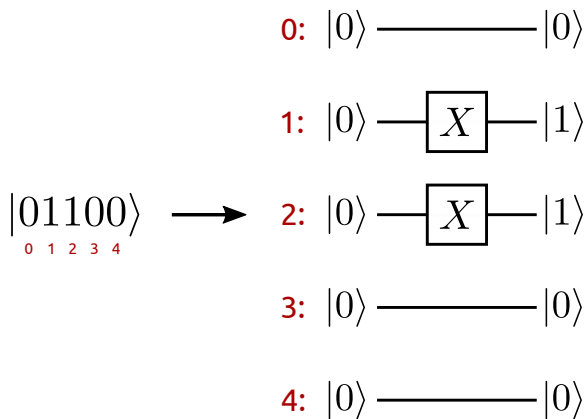
The tensor product is linear and distributive. Given

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\varphi\rangle = \gamma|0\rangle + \delta|1\rangle,$$

Question: Are tensor products of states still normalized?

Qubit ordering (very important!)

In PennyLane:



This is different in other frameworks.

Multi-qubit gates

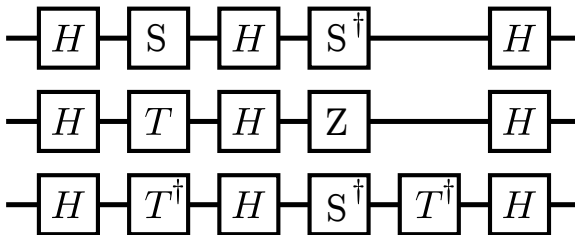
Unitary operations also compose under tensor product.

Example: apply U_1 to $|\psi\rangle$ and U_2 to $|\varphi\rangle$

Exercise: determine the state of a 3-qubit system if H is applied to qubit 0, X and then S are applied to qubit 1.

Multi-qubit gates

The circuits we've seen so far only involve single-qubit gates:



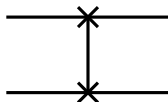
Surely this isn't all we can do...

Image credit: Xanadu Quantum Codebook I.11

SWAP

Define an operation that swaps the state of two qubits. First, specify action on basis states:

Circuit element:

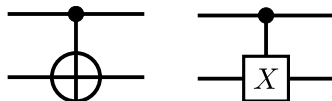


PennyLane: `qp.SWAP`

CNOT

CNOT = “controlled-NOT”. A NOT (X) is applied to second qubit only if first qubit is in state $|1\rangle$.

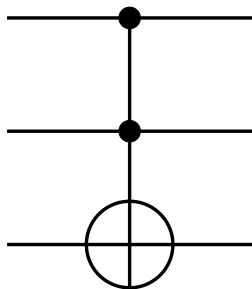
Circuit elements:



PennyLane: `qp.CNOT`

Toffoli (TOF)

There are also gates on more than two qubits, like the Toffoli gate, which is a controlled-CNOT, or controlled-controlled-NOT.



$$TOF|000\rangle =$$

$$TOF|001\rangle =$$

$$TOF|010\rangle =$$

$$TOF|011\rangle =$$

$$TOF|100\rangle =$$

$$TOF|101\rangle =$$

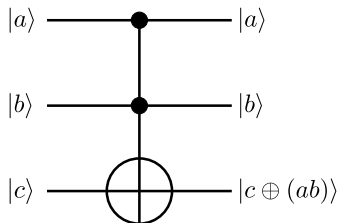
$$TOF|110\rangle =$$

$$TOF|111\rangle =$$

PennyLane: `qp.Toffoli`

Quantum circuits and Boolean functions

The Toffoli implements a reversible AND gate. It is also universal for classical reversible computing.

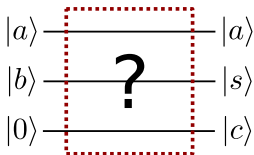


Exercise: what Boolean function does CNOT implement?

Quantum circuits and Boolean functions

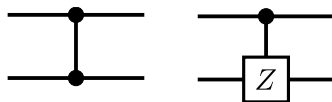
X, CNOT, TOF can be used to create Boolean arithmetic circuits.

Exercise: use a combination of these gates to create a quantum version of a one-bit adder. Test it in PennyLane on all possible bit values of a and b .



Example: controlled-Z (CZ)

Exercise: what is the matrix representation of this operation?



PennyLane: `qp.CZ`

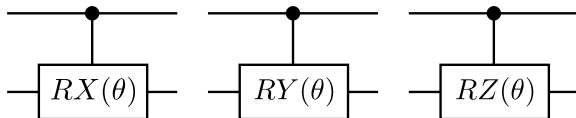
Image credit: Codebook node I.13

Example: controlled rotations (RX , RY , RZ)

We can control arbitrary Pauli rotations:

$$CRY(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ 0 & 0 & \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}$$

Circuit elements:



PennyLane: `qp.CRX`, `qp.CRY`, `qp.CRZ`

Image credit: Codebook node I.14

Controlled- U

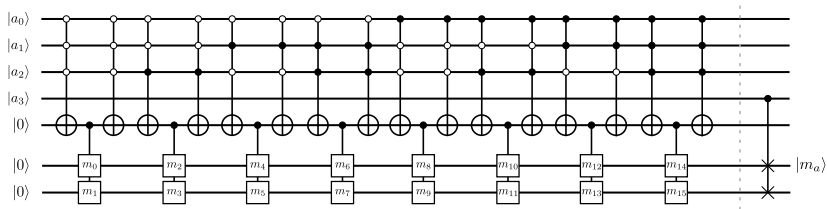
There is a pattern here:

$$CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad CRY(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ 0 & 0 & \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}$$

More generally,

Controlled unitary operations

Any unitary operation can be turned into a controlled operation, controlled on any state.



Most common controls are controlled-on- $|1\rangle$ (filled circle), and controlled-on- $|0\rangle$ (empty circle).

Controlled unitary operations with `qp.ctrl`

Remember `qp.adjoint`:

```
@qp.qnode(dev)
def my_circuit():
    qp.S(wires=0)
    qp.adjoint(qp.S)(wires=0)
    return qp.sample()
```

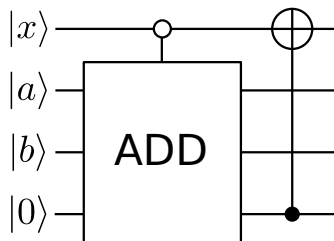
A similar *transform* allows us to perform arbitrary controlled operations (or entire quantum functions)!

```
@qp.qnode(dev)
def my_circuit():
    qp.S(wires=0)
    qp.ctrl(qp.S, control=1)(wires=0)
    return qp.sample()
```

Controlled unitary operations with `qp.ctrl`

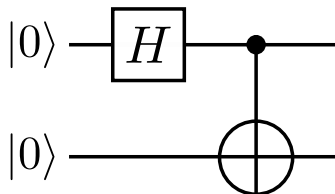
Exercise: using your adder from earlier, implement a circuit in PennyLane that uses `qp.ctrl` to

- perform addition if the control qubit is in state $|0\rangle$
- flip the state of the control qubit if the carry bit is 1



Controlling on superpositions

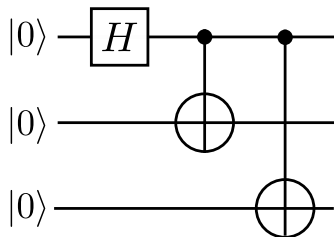
Exercise: What does CNOT do when the control qubit is in a superposition? Determine the combined output state of this circuit, then try to express it as a tensor product of two single-qubit states.



Entanglement

CNOT is an **entangling gate**!

Entanglement generalizes to more than two qubits:



Exercise: Express the output state of this circuit in the computational basis.

Next time

Content:

- Multi-qubit measurement and the Bell basis
- Superdense coding

Action items:

- 1 Prepare for Quiz 3
- 2 Work on practice assignment 1
- 3 Work through exercises from tutorials

Recommended reading:

- Codebook nodes IQC, SQ, MQ; Nielsen & Chuang 1.2, 1.3.1-1.3.7, 2.2.3-2.2.5, 2.3, 4.3